Anteproyecto de Trabajo de Fin de Grado

Carlos Jiménez Crespo

20 de septiembre de 2026

Autor: Carlos Jiménez Crespo

Tutora: Elena Katia Leal Algara

Titulación: Grado en Ingeniería Informática

Título: JoularJX: Análisis y evaluación de una herramienta para la medición del consumo energético del software

Departamento: Departamento de Automática

1. Introducción

El consumo energético del software es una problemática de creciente relevancia en el contexto de la sostenibilidad digital. Aunque se han producido avances significativos en la eficiencia energética del hardware, el software raramente se diseña o evalúa teniendo en cuenta su huella energética. Esta situación tiene un impacto real en el consumo de los dispositivos de usuario y en la infraestructura de cómputo global [1].

En este contexto, JoularJX es un agente Java de código abierto, desarrollado en la Universidad de Pau (Francia) por el Dr. Adel Noureddine, que permite medir el consumo energético de aplicaciones Java a nivel de método de forma no intrusiva, sin modificar el código fuente de la aplicación monitorizada [2]. JoularJX se engancha a la JVM en el arranque y, mediante muestreo estadístico del *stacktrace* activo, atribuye a cada método la energía consumida por el proceso durante su ejecución. Es compatible con Windows, Linux y macOS, y está activamente mantenida (última versión: 3.1.0, febrero de 2026) [3].

Este Trabajo Fin de Grado propone estudiar JoularJX en profundidad y aplicarla a un caso de experimentación concreto: la comparación del consumo energético de Jackson [4] y Gson [5], las dos librerías Java de procesamiento de JSON más utilizadas. Ambas resuelven el mismo problema funcional (serializar y deserializar objetos Java a formato JSON), pero con arquitecturas internas radicalmente distintas, lo que las convierte en un caso de estudio ideal para analizar cómo las decisiones de diseño del software impactan en su eficiencia energética [6].

2. Objetivos

Objetivo general

Analizar y evaluar JoularJX como herramienta para la medición del consumo energético de aplicaciones Java, aplicándola a la comparación de dos librerías de código abierto ampliamente utilizadas que resuelven el mismo problema funcional.

Objetivos específicos

Los objetivos específicos de este proyecto son los siguientes:

- Comprender cómo mide JoularJX el consumo energético a nivel de método Java: arquitectura de agente, muestreo de *stacktrace*, fuentes de datos (la interfaz Intel RAPL expuesta por el subsistema `powercap` del núcleo Linux) y formatos de salida.
- Instalar y configurar JoularJX en un entorno GNU/Linux, documentando el proceso paso a paso, incluida la verificación de la disponibilidad de los contadores RAPL y los permisos de lectura sobre `/sys/class/powercap`.
- Preparar un banco de pruebas reproducible sobre las librerías Jackson y Gson que permita medir su consumo energético en operaciones equivalentes de serialización y deserialización de JSON.
- Ejecutar las mediciones con JoularJX, analizar los resultados (consumo total, consumo por método y *hotspots* energéticos) y comparar ambas librerías.
- Evaluar las limitaciones de JoularJX: precisión del muestreo, *overhead* del agente y variabilidad de las mediciones entre ejecuciones.

Campo de aplicación

El TFG se enmarca en el área de ingeniería del software sostenible. Los resultados son de interés para desarrolladores Java, arquitectos de software y cualquier organización que quiera tomar decisiones informadas sobre qué librerías usar en función de su eficiencia energética. El trabajo no está sujeto a ningún acuerdo de confidencialidad.

3. Descripción del trabajo

El trabajo se estructura en tres bloques: estudio teórico de JoularJX, configuración del entorno experimental y experimentación comparativa entre Jackson y Gson. Los componentes principales son los siguientes:

1. **Estudio de JoularJX.** Revisión de los artículos originales de JoularJX [2], [3], estudio de su arquitectura interna como agente Java (API `java.lang.instrument`), mecanismo de muestreo de *stacktrace*, fuentes de energía según la plataforma y formato de salida CSV.

2. **Instalación y configuración del entorno.** Instalación de JoularJX (*jar* del agente) sobre GNU/Linux, comprobación de que el procesador expone los contadores RAPL a través del subsistema `powercap`, ajuste de los permisos de lectura de los ficheros `energy_uj` y configuración del fichero `config.properties`. Verificación del funcionamiento con un programa Java de prueba. Descarga de Jackson [4] y Gson [5] desde Maven Central.

3. **Diseño del banco de pruebas.** Diseño de un conjunto de casos de prueba equivalentes para ambas librerías: serialización y deserialización de objetos simples y complejos, con diferentes tamaños de *payload* (pequeño, mediano y grande). Uso de JMH (*Java Microbenchmark Harness*) para garantizar condiciones de medición estables y reproducibles.

4. **Experimentación y análisis.** Ejecución de JoularJX sobre ambas librerías. Recogida de datos de consumo total (julios), consumo por método y evolución temporal. Repetición de las mediciones para verificar su estabilidad. Análisis de resultados e identificación de *hotspots* energéticos. Comparación normalizada por operación (julios por serialización y deserialización).

4. Metodología y plan de trabajo

El trabajo seguirá un enfoque incremental, con las siguientes etapas, orientadas a la consecución de los objetivos descritos en la sección 2:

1. Lectura de los artículos sobre JoularJX y de la literatura sobre consumo energético del software.

2. Instalación de JoularJX sobre GNU/Linux, comprobación del acceso a los contadores RAPL y verificación del entorno con un programa de prueba.

3. Estudio de las arquitecturas de Jackson y Gson, y diseño del banco de pruebas con JMH.

4. Ejecución de las mediciones con JoularJX, repetición para verificar su estabilidad y análisis de los resultados.

5. Comparación normalizada, evaluación de las limitaciones de JoularJX y extracción de conclusiones.

6. Redacción de la memoria final del TFG.

El diagrama de Gantt asociado a esa planificación, para la convocatoria de junio/julio de 2027, es el mostrado en la figura 2.

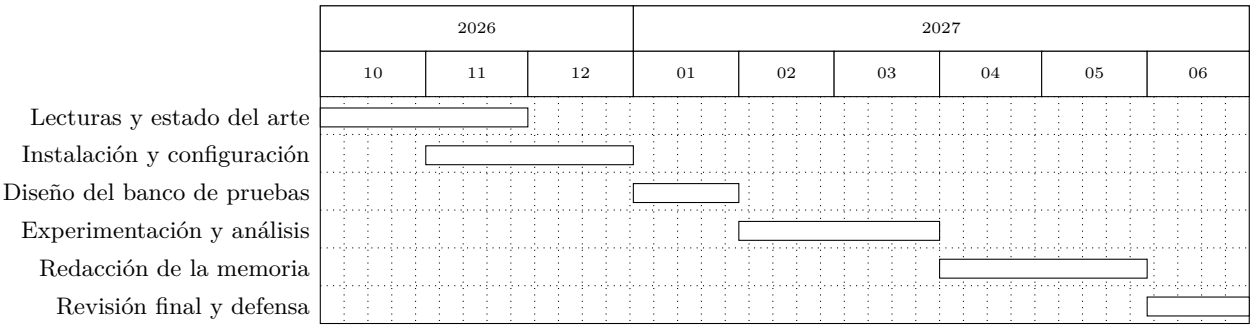


Figura 2: Diagrama de Gantt de la planificación del trabajo.

5. Medios

Las herramientas y recursos necesarios para desarrollar este proyecto son los siguientes:

- Ordenador personal con sistema operativo GNU/Linux (Ubuntu) y procesador Intel o AMD (a partir de Ryzen) con soporte de RAPL.
- JDK 11 o superior y Apache Maven.
- JoularJX (agente Java, licencia GPL-3.0) [7].
- Jackson Databind (FasterXML, Apache 2.0) [4] y Gson (Google, Apache 2.0) [5], disponibles en Maven Central.
- Procesador de textos LaTeX para la redacción de la memoria.
- Repositorio Git para el control de versiones del código y de la documentación.
- Acceso a las bases de datos bibliográficas de la UAH (IEEE Xplore, ACM Digital Library y ScienceDirect).

No se requieren medios hardware específicos ni financiación adicional.

Referencias

[1] R. Verdecchia, P. Lago, C. Ebert y C. de Vries, «Green IT and Green Software», *IEEE Software*, vol. 38, n.º 6, págs. 7-15, 2021.

[2] A. Noureddine, «PowerJoular and JoularJX: Multi-Platform Software Power Monitoring Tools», en *Proceedings of the 18th International Conference on Intelligent Environments (IE2022)*, Biarritz, France, 2022.

[3] A. Noureddine, «JoularJX 3: In-Depth Green Software Analysis Everywhere», en *Proceedings of the 11th International Conference on ICT for Sustainability (ICT4S 2025)*, Dublin, Ireland, 2025.

[4] FasterXML. «Jackson Project». (), dirección: https://github.com/FasterXML/jackson.

[5] Google. «Gson». (), dirección: https://github.com/google/gson.

[6] A. Noureddine y O. Le Goaër, «Investigating the Impact of Software Design Patterns on Energy Consumption», en *Proceedings of the 22nd IEEE International Conference on Software Architecture (ICSA 2025)*, Odense, Denmark, 2025.

[7] A. Noureddine. «JoularJX». (), dirección: https://github.com/joular/joularjx.